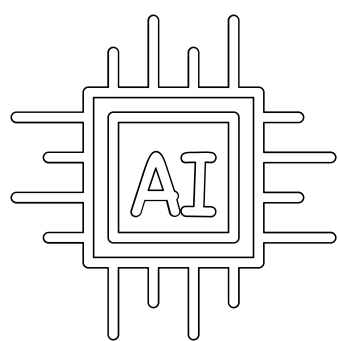


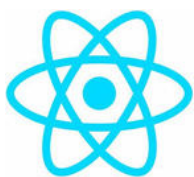


TUMORSENSE

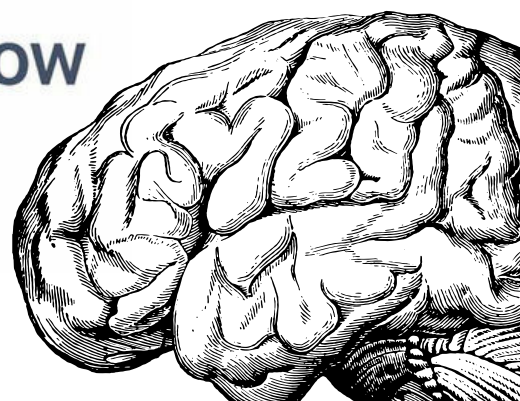
Introduction:

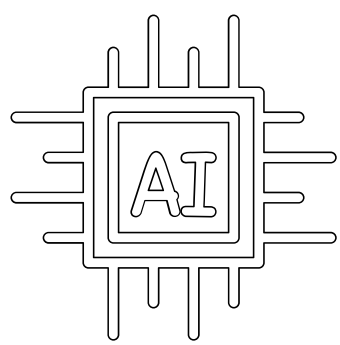
TumorSense is an AI-powered web application developed using React and Vite, designed to assist in the early detection of brain tumors through MRI image classification. The platform integrates a deep learning model hosted on Render.com to accurately categorize MRI scans into four distinct tumor types. With a clean and responsive interface, the website—hosted on Vercel—provides users with a fast, seamless experience

Frontend (Web)	React + Vite (hosted on Vercel)
Backend	Flask (Python), Render Web Service
ML Model	TensorFlow, Teachable Machine (TFLite)
Image Uploads	Cloudinary



TensorFlow





TUMORSENSE

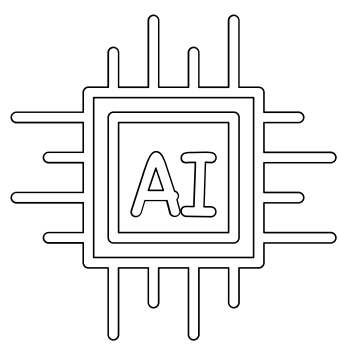
Why is this project important in the real world?

Brain tumors are abnormal growths in the brain that can interfere with essential neurological functions such as memory, vision, balance, and coordination. They are often classified as benign or malignant, and timely diagnosis plays a crucial role in determining treatment success and patient survival rates.

Magnetic Resonance Imaging (MRI) is the standard method for detecting and analyzing brain tumors. However, interpreting MRI scans accurately requires significant medical expertise, which may not always be accessible, especially in remote or underserved regions.

This is where **AI** can make a meaningful impact. By automating MRI scan analysis with deep learning, tools like **TumorSense** aim to assist healthcare professionals with faster, consistent, and more accessible brain tumor detection—supporting early intervention and better outcomes.





TUMORSENSE

System Architecture & Flow:

Upload MRI Image

The user uploads a brain MRI scan through the TumorSense web interface built with React.

Image Sent to AI Backend

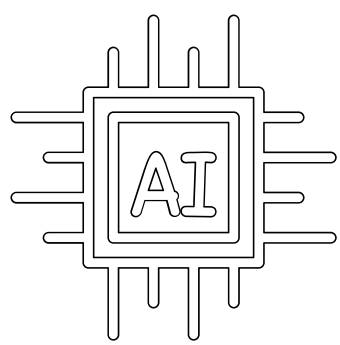
The uploaded image is sent to a Flask API hosted on Render.com, which forwards it to a trained deep learning model.

Tumor Classification by Model

The backend preprocesses the image and the AI model classifies it into one of four categories: Glioma, Meningioma, Pituitary, or No Tumor.

Result Displayed to User





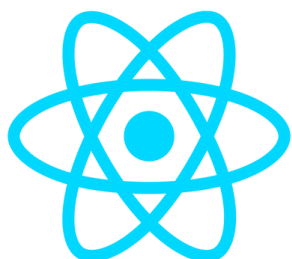
TUMORSENSE

The Frontend:

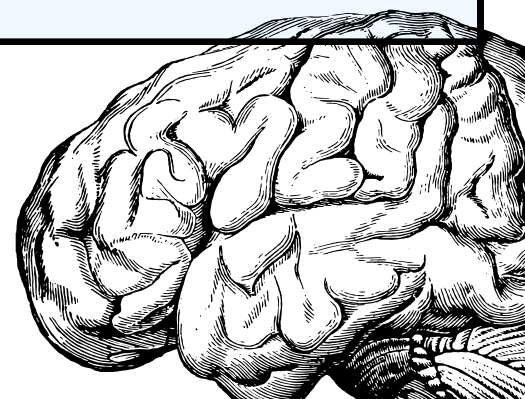
The frontend is built with React and bundled using Vite for fast loading and smooth development. It provides a simple interface where users can upload MRI images. Upon upload, the app sends the image to the backend API and displays the predicted tumor type instantly. The frontend is hosted on Vercel, ensuring high availability and fast response times.

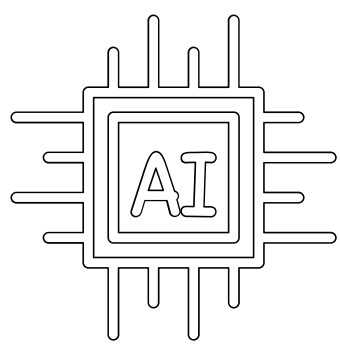
REACT

React is a popular JavaScript library developed by Facebook for building fast, responsive user interfaces using reusable components. In TumorSense, React manages the upload of MRI images, communicates with the backend API, and dynamically displays tumor classification results without page reloads. Its component-based design keeps the code organized and scalable, while the virtual DOM ensures smooth performance across devices.



React





TUMORSENSE

VITE

Vite is a modern build tool that provides fast development and optimized bundling for web applications. It speeds up the development process with instant server start and hot module replacement, ensuring quick updates during coding. For TumorSense, Vite helps deliver a fast-loading frontend with smooth user interactions.

VERCEL

Vercel is a cloud platform for hosting web applications, offering global content delivery for fast and reliable access. It enables seamless deployment directly from the GitHub repository with automatic updates. Hosting TumorSense on Vercel ensures users get a quick, accessible, and scalable web experience.



FRONTEND SHOWCASE

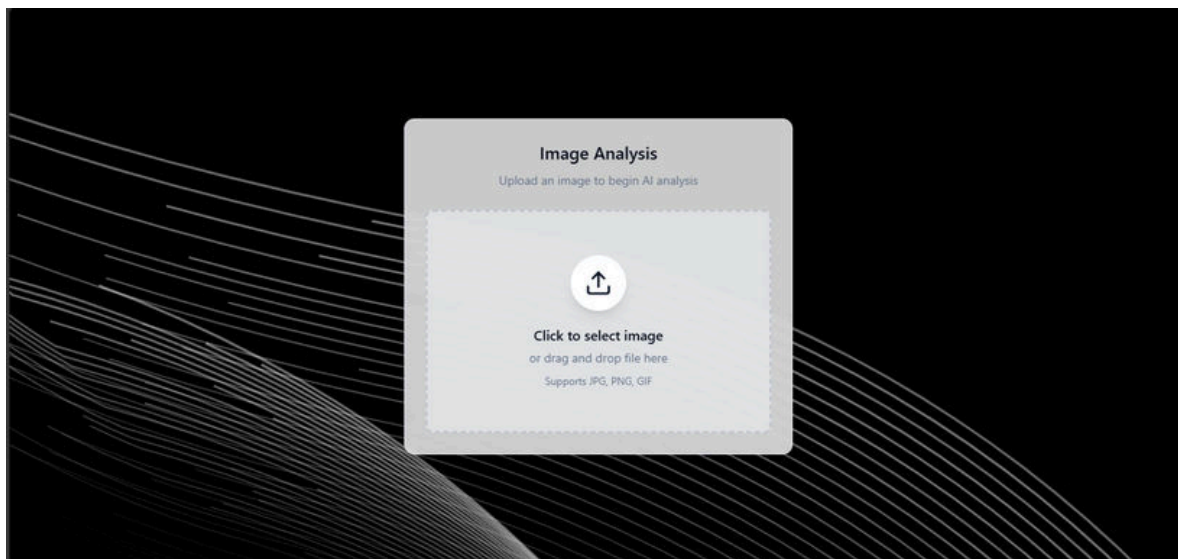
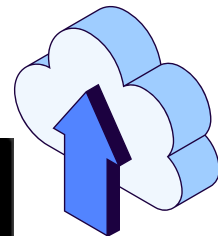


IMAGE UPLOAD

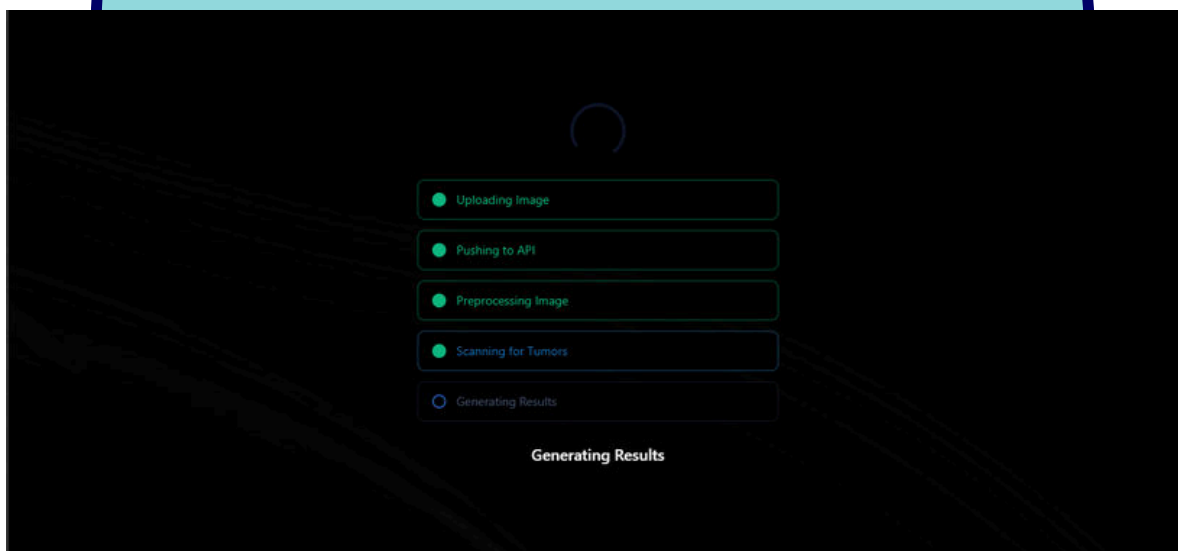
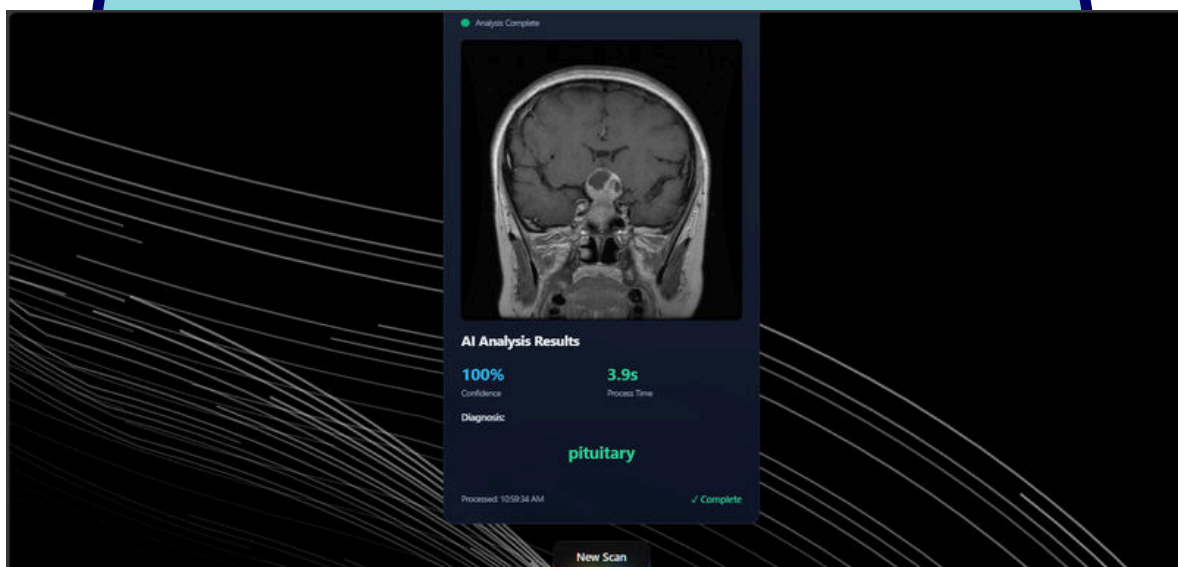
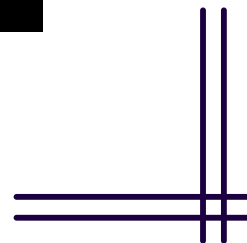
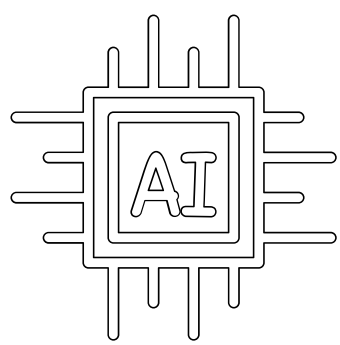


IMAGE PROCESSING AND MODEL PREDICTION



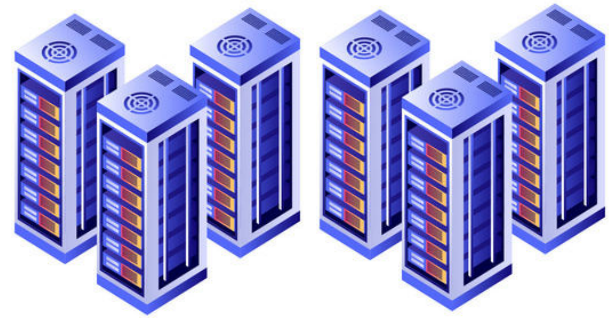
RESULTS





TUMORSENSE

THE BACKEND:



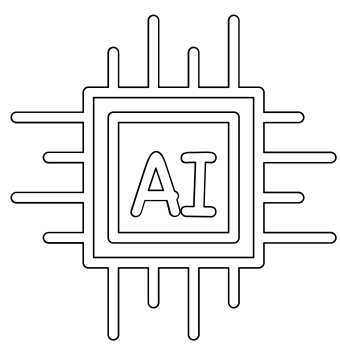
Framework and Hosting:

The backend of TumorSense is built using Flask, a lightweight and easy-to-use Python web framework that handles API requests and communication with the frontend. To run the Flask app efficiently in a production environment, Gunicorn is used as the application server, managing multiple requests smoothly. The entire backend is hosted on Render.com, a cloud platform that provides reliable hosting, automatic scaling, and seamless deployment, ensuring the API is always available to users.

FLASK



Flask is a lightweight Python web framework used to build the backend API of TumorSense. It provides the necessary tools to handle HTTP requests, manage routing, and process data sent from the frontend. Flask's simplicity and flexibility make it ideal for developing APIs quickly and efficiently. In TumorSense, Flask handles uploading MRI images, running them through the AI model, and sending predictions back to the frontend.



TUMORSENSE

GUNICORN

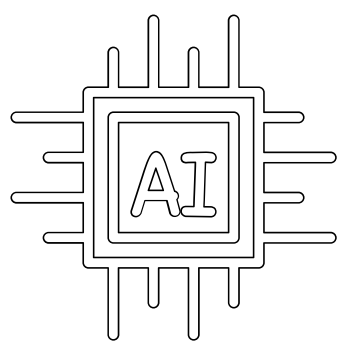
Gunicorn (Green Unicorn) is a Python WSGI HTTP server designed to run Python web applications in production. Unlike Flask's built-in server, Gunicorn can manage multiple simultaneous requests and improve performance and reliability. It acts as a bridge between the web server and the Flask application, efficiently distributing incoming API requests. TumorSense uses Gunicorn on Render.com to ensure the backend API remains responsive under load.



RENDER

Render.com is a cloud hosting platform that simplifies deploying and managing web applications and APIs. It offers features like automatic scaling, continuous deployment from GitHub repositories, and easy configuration. Hosting the TumorSense backend on Render.com ensures the API is always accessible, can handle varying traffic, and is easy to update with new model versions or bug fixes.





TUMORSENSE

RENDER BUILD PROCESS

Build & Deploy

Repository

The repository used for your Web Service.

`https://github.com/GG-113131/API`

[Edit](#)

Branch

The Git branch to build and deploy.

`gg`

[Edit](#)

Build Command

Render runs this command to build your app before each deploy.

```
$ pip install -r requirements.txt
```

[Edit](#)

Pre-Deploy Command Optional

Render runs this command before the start command. Useful for database migrations and static asset uploads.

`$`

[Edit](#)

Start Command

Render runs this command to start your app with each deploy.

```
$ gunicorn --bind 0.0.0.0:5000 app:app --timeout 90
```

[Edit](#)

Build Logs:

on your machine, GPU will not be used.

Jun 5 11:28:50 AM hch5s 2025-06-05 07:28:50.948929: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.

Jun 5 11:28:50 AM hch5s 2025-06-05 07:28:50.958551: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

Jun 5 11:28:50 AM hch5s To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

Jun 5 11:29:30 AM hch5s 2025-06-05 07:29:30.881502: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT

Jun 5 11:30:47 AM hch5s INFO: Created TensorFlow Lite XNNPACK delegate for CPU.

Jun 5 11:30:54 AM hch5s [2025-06-05 07:30:54 +0000] [65] [INFO] Starting gunicorn 23.0.0

Jun 5 11:30:54 AM hch5s [2025-06-05 07:30:54 +0000] [65] [INFO] Listening at: <http://0.0.0.0:5000> (65)

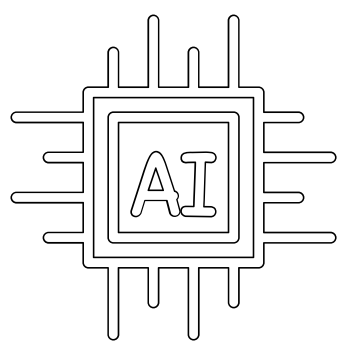
Jun 5 11:30:54 AM hch5s [2025-06-05 07:30:54 +0000] [65] [INFO] Using worker: sync

Jun 5 11:30:54 AM hch5s [2025-06-05 07:30:54 +0000] [81] [INFO] Booting worker with pid: 81

Jun 5 11:31:00 AM hch5s Downloading model from Google Drive...

Jun 5 11:31:00 AM hch5s Model downloaded successfully.

Jun 5 11:31:00 AM hch5s 127.0.0.1 - - [05/Jun/2025:07:31:00 +0000] "POST /predict HTTP/1.1" 200 52 "-" "curl/8.5.0"



TUMORSENSE



Files Deployed On Render.com (Through Github Repo.)

📄 app.py

Update app.py

📄 labels.txt

Add files via upload

📄 requirements.txt

Update requirements.txt

◆ APP.PY

This is the main Flask application file. It defines the API routes, handles image uploads, loads the trained model, and returns tumor predictions as a response.

◆ LABELS.TXT

A simple text file containing the names of the tumor classes (e.g., glioma, meningioma, pituitary, no_tumor). The model's output index is mapped to these labels for readable predictions.

◆ REQUIREMENTS.TXT

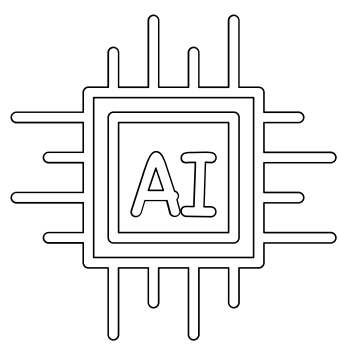
Lists all the Python libraries needed to run the backend (e.g., Flask, TensorFlow, NumPy). Render uses this file to automatically install dependencies during deployment.

```
flask
flask-cors
tensorflow==2.12.0
pillow
requests
gunicorn
```

REQUIREMENTS.TXT

```
0 glioma
1 meningioma
2 notumor
3 pituitary
```

LABELS.TXT

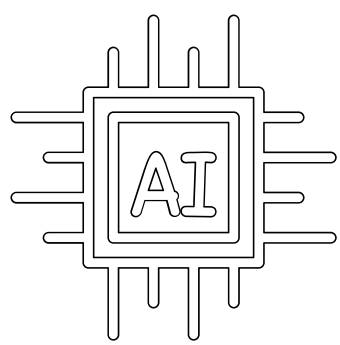


TUMORSENSE



APP.PY

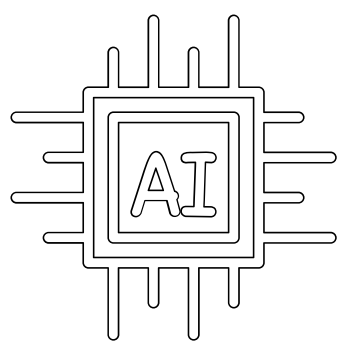
```
1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3 from PIL import Image, ImageOps
4 import numpy as np
5 import requests
6 from io import BytesIO
7 import os
8 import tensorflow as tf
9
10 # Constants
11 MODEL_FILE_ID = "1gC8fcawwxmi0eVu1AT9qx_q2SW2Lhbah"
12 MODEL_URL = f"https://drive.google.com/uc?export=download&id={MODEL_FILE_ID}"
13 MODEL_PATH = "tumor_model.tflite"
14
15 # Download model if it doesn't exist
16 def download_model():
17     if not os.path.exists(MODEL_PATH):
18         print("📄 Downloading model from Google Drive ...")
19         r = requests.get(MODEL_URL, stream=True)
20         with open(MODEL_PATH, "wb") as f:
21             for chunk in r.iter_content(chunk_size=8192):
22                 if chunk:
23                     f.write(chunk)
24         print("✅ Model downloaded successfully.")
25
```



TUMORSENSE



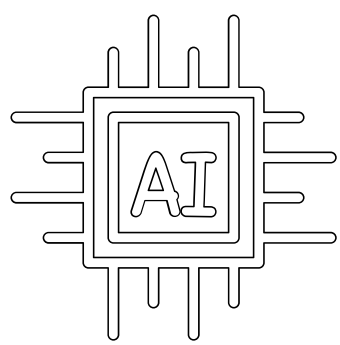
```
1  # Download model before loading
2  download_model()
3
4  # Load class labels
5  with open("labels.txt", "r") as f:
6      class_names = [line.strip().split(" ", 1)[1] for line in f.readlines()]
7
8  # Load TFLite model and allocate tensors
9  interpreter = tf.lite.Interpreter(model_path=MODEL_PATH)
10 interpreter.allocate_tensors()
11
12 # Get input and output details
13 input_details = interpreter.get_input_details()
14 output_details = interpreter.get_output_details()
15
16 # Initialize Flask app
17 app = Flask(__name__)
18 CORS(app)
19
20 @app.route("/predict", methods=["POST"])
21 def predict():
22     try:
23         data = request.get_json()
24         image_url = data.get("image_url")
25         if not image_url:
26             return jsonify({"error": "Missing 'image_url' in request"}), 400
27
```

TUMORSENSE



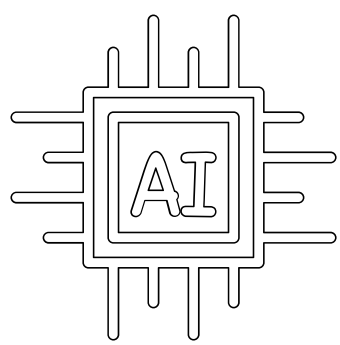
```
1  # Download and preprocess the image
2  response = requests.get(image_url)
3  image = Image.open(BytesIO(response.content)).convert("RGB")
4  size = (299, 299)
5  image = ImageOps.fit(image, size, Image.Resampling.LANCZOS)
6  image_array = np.asarray(image)
7
8  # ✅ Correct preprocessing: scale to [0, 1]
9  normalized_image_array = image_array.astype(np.float32) / 255.0
10 input_data = np.expand_dims(normalized_image_array, axis=0).astype(np.float32)
11
12 # Set input tensor
13 interpreter.set_tensor(input_details[0]['index'], input_data)
14
15 # Run inference
16 interpreter.invoke()
17
18 # Get output and return result
19 output_data = interpreter.get_tensor(output_details[0]['index'])
20 index = np.argmax(output_data)
21 predicted_class = class_names[index]
22 confidence = float(output_data[0][index])
23
```

TUMORSENSE



```
1         return jsonify({
2             "class": predicted_class,
3             "confidence": confidence
4         })
5
6     except Exception as e:
7         return jsonify({"error": str(e)}), 500
8
9 if __name__ == "__main__":
10     print("✅ Flask server running on http://localhost:5000")
11     app.run(port=5000)
12
```



TUMORSENSE

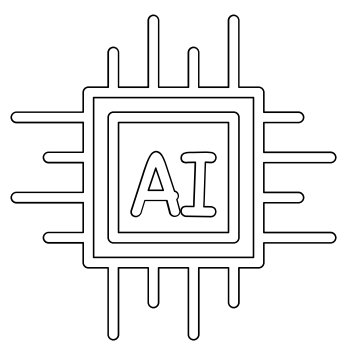


APP.PY Explained

This Flask-based web app serves as a brain tumor classification API using a TensorFlow Lite model. It performs the following key tasks:

- Downloads the TFLite model from Google Drive if it's not already available locally.
- Loads class labels from a labels.txt file.
- Sets up a /predict API endpoint that:
 - Accepts an image URL in JSON format.
 - Downloads and resizes the image to 299x299 pixels.
 - Preprocesses it for model inference (normalizes pixel values).
 - Runs the image through the TFLite model.
 - Returns the predicted class and confidence as a JSON response.

CORS is enabled for cross-origin requests, and the server runs locally on port 5000.



TUMORSENSE

Model Training

DATASET & PREPROCESSING

- The dataset is the Masoud Nickparvar Brain Tumor MRI Dataset from Kaggle, consisting of MRI images labeled into 4 classes:

meningioma

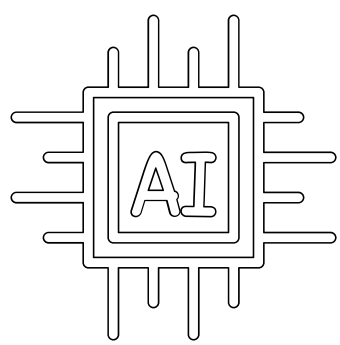
glioma

pituitary

no tumor

The Kaggle logo, which consists of the word 'kaggle' in a blue, lowercase, sans-serif font.

- Images are loaded from folder structures into Pandas DataFrames with columns for image paths and corresponding labels.
- The dataset is split into:
 - Training set**
 - Validation set (split from original test data)**
 - Test set**
- Image augmentation is applied only during training with:
 - Rescaling pixel values to $[0, 1]$
 - Random brightness adjustments between 0.8 and 1.2
 - This helps improve model robustness.
- All images are resized to 299x299 pixels to match Xception's input size.



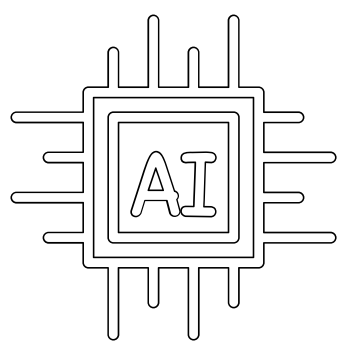
TUMORSENSE

MODEL ARCHITECTURE

- Utilized Xception model pretrained on ImageNet as a base for transfer learning, excluding its top classification layers.
- Added custom top layers:
- Flatten layer to convert features to 1D
- Dropout layers to prevent overfitting (0.3 and 0.25 rates)
- Dense layer with 128 neurons (ReLU activation)
- Final Dense layer with 4 neurons and softmax activation for multi-class classification
- Compiled model with:
- Adamax optimizer (learning rate = 0.001)
- Categorical cross-entropy loss (for multi-class)
- Metrics: accuracy, precision, recall

TRAINING

- Model trained for 10 epochs on the training set with validation on the validation set.
- Batch size set to 32 during training and validation; 16 for test data to manage memory.
- Training progress includes accuracy, loss, precision, and recall plotted to visualize overfitting/underfitting and best-performing epochs.



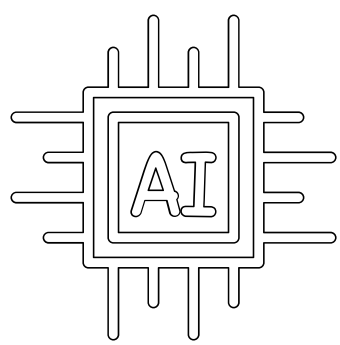
TUMORSENSE

EVALUATION & TESTING

- Final model evaluated on training, validation, and test datasets, reporting loss and accuracy.
- Generated a confusion matrix and detailed classification report (precision, recall, F1-score) to analyze class-wise performance.
- Included a prediction function to load an individual image and output predicted class probabilities with a visualization bar plot.

MODEL SAVING & CONVERSION

- Saved the trained Keras model to a file (AI_MRI_TumorDetect.keras).
- Converted the Keras model to TensorFlow Lite format (tumor_model.tflite) for deployment on Render.com.
- Generated a labels.txt file to map class indices to class names, which is useful for interpreting predictions in deployment.



TUMORSENSE

GLOSSARY OF TECHNICAL TERMS

Transfer Learning

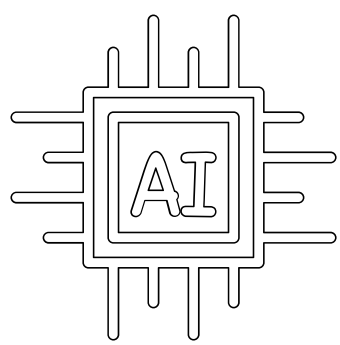
A technique in machine learning where a model developed for one task is reused as the starting point for a different but related task. Instead of training a deep learning model from scratch (which requires huge amounts of data and time), transfer learning uses a pretrained model—often trained on large datasets like ImageNet—and fine-tunes it on a smaller, specific dataset. This helps the model learn features faster and improves performance, especially when you don't have a large dataset.

Epochs

One epoch means the model has processed the entire training dataset once. Training usually involves multiple epochs to allow the model to learn and improve its accuracy by updating weights multiple times.

Keras

A user-friendly, high-level neural networks API written in Python. It runs on top of TensorFlow and simplifies building, training, and testing deep learning models.



TUMORSENSE

GLOSSARY OF TECHNICAL TERMS

Rescaling

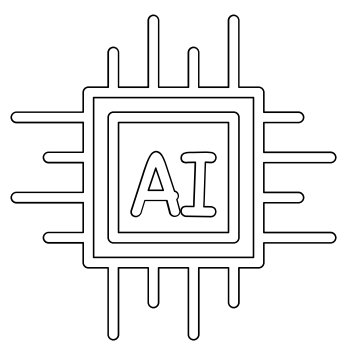
A preprocessing step where image pixel values (originally ranging from 0 to 255) are normalized to a smaller range, typically 0 to 1, by dividing by 255. This helps models train more efficiently and converge faster.

Xception Model

A powerful convolutional neural network architecture based on “extreme” inception modules using depthwise separable convolutions. It’s often used as a base model in transfer learning for image classification due to its accuracy and efficiency.

Pretrained Weights

Weights saved from a model previously trained on a large dataset (like ImageNet). These weights contain learned features useful for many image-related tasks, enabling transfer learning by starting training with these “learned knowledge” instead of random initialization.



TUMORSENSE

GLOSSARY OF TECHNICAL TERMS

Precision and Recall

Precision: The ratio of correctly predicted positive results to the total predicted positives. It answers, “Of all the predicted positives, how many were actually positive?”

Recall: The ratio of correctly predicted positive results to all actual positives. It answers, “Of all actual positives, how many did the model correctly find?”

Confusion Matrix:

A table used to measure the performance of a classification model by showing the actual versus predicted classifications. It includes counts of true positives, false positives, true negatives, and false negatives for each class.

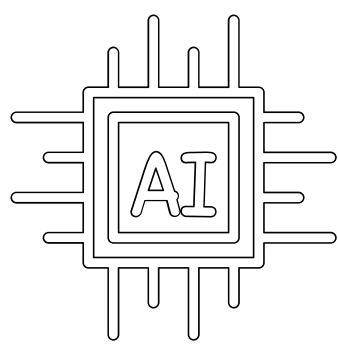
Train/Test/Validation Split

The practice of dividing your dataset into three parts:

Training set: Used to train the model.

Validation set: Used during training to tune hyperparameters and check model performance to prevent overfitting.

Test set: Used after training to evaluate final model performance on unseen data



TUMORSENSE

MODEL TRAINING CODE

```
import kagglehub

masoudnickparvar_brain_tumor_mri_dataset_path = kagglehub.dataset_download(
    "masoudnickparvar/brain-tumor-mri-dataset"
)

print("Data source import complete.")

"""# 1. Import needed libraries"""

import os
from PIL import Image
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from glob import glob

# -----

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# -----

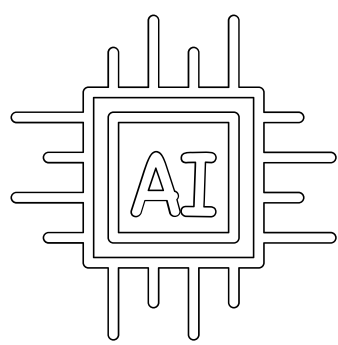
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras.metrics import Precision, Recall
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
import warnings

warnings.filterwarnings("ignore")

"""# 2. Preprocessing

## 2.1 Load data
"""
```



TUMORSENSE

MODEL TRAINING CODE

```
def train_df(tr_path):
    classes, class_paths = zip(
        *[
            (label, os.path.join(tr_path, label, image))
            for label in os.listdir(tr_path)
            if os.path.isdir(os.path.join(tr_path, label))
            for image in os.listdir(os.path.join(tr_path, label))
        ]
    )

    tr_df = pd.DataFrame({"Class Path": class_paths, "Class": classes})
    return tr_df

def test_df(ts_path):
    classes, class_paths = zip(
        *[
            (label, os.path.join(ts_path, label, image))
            for label in os.listdir(ts_path)
            if os.path.isdir(os.path.join(ts_path, label))
            for image in os.listdir(os.path.join(ts_path, label))
        ]
    )

    ts_df = pd.DataFrame({"Class Path": class_paths, "Class": classes})
    return ts_df

tr_df = train_df("/kaggle/input/brain-tumor-mri-dataset/Training")

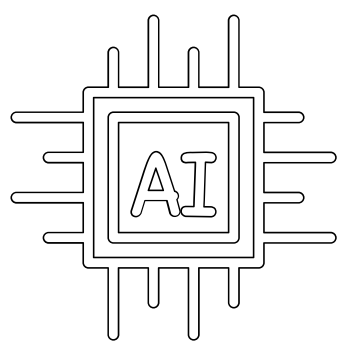
tr_df
ts_df = test_df("/kaggle/input/brain-tumor-mri-dataset/Testing")

ts_df

# Count of images in each class in train data

plt.figure(figsize=(15, 7))
ax = sns.countplot(data=tr_df, y=tr_df["Class"])

plt.xlabel("")
plt.ylabel("")
plt.title("Count of images in each class", fontsize=20)
ax.bar_label(ax.containers[0])
plt.show()
```



TUMORSENSE

MODEL TRAINING CODE

```
# Count each class in test data

plt.figure(figsize=(15, 7))
ax = sns.countplot(y=ts_df["Class"], palette="viridis")

ax.set(xlabel="", ylabel="", title="Count of images in each class")
ax.bar_label(ax.containers[0])

plt.show()

"""## 2.2 Split data into train, test, valid"""

valid_df, ts_df = train_test_split(
    ts_df, train_size=0.5, random_state=20, stratify=ts_df["Class"]
)

valid_df

"""## 2.3 Data preprocessing"""

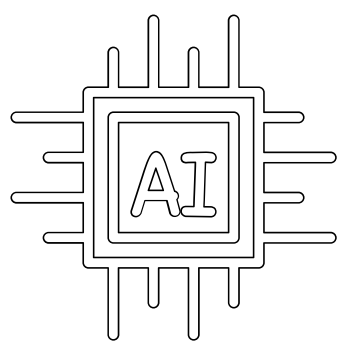
batch_size = 32
img_size = (299, 299)

_gen = ImageDataGenerator(rescale=1 / 255, brightness_range=(0.8, 1.2))

ts_gen = ImageDataGenerator(rescale=1 / 255)

tr_gen = _gen.flow_from_dataframe(
    tr_df,
    x_col="Class Path",
    y_col="Class",
    batch_size=batch_size,
    target_size=img_size,
)

valid_gen = _gen.flow_from_dataframe(
    valid_df,
    x_col="Class Path",
    y_col="Class",
    batch_size=batch_size,
    target_size=img_size,
)
```

TUMORSENSE

MODEL TRAINING CODE

```
ts_gen = ts_gen.flow_from_dataframe(
    ts_df,
    x_col="Class Path",
    y_col="Class",
    batch_size=16,
    target_size=img_size,
    shuffle=False,
)

"""## 2.4 Getting samples from data"""

class_dict = tr_gen.class_indices
classes = list(class_dict.keys())
images, labels = next(ts_gen)

plt.figure(figsize=(20, 20))

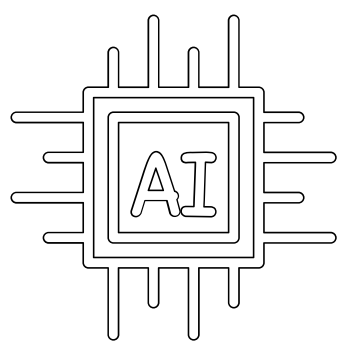
for i, (image, label) in enumerate(zip(images, labels)):
    plt.subplot(4, 4, i + 1)
    plt.imshow(image)
    class_name = classes[np.argmax(label)]
    plt.title(class_name, color="k", fontsize=15)
plt.show()

"""# 3. Building Deep Learning Model"""

img_shape = (299, 299, 3)
base_model = tf.keras.applications.Xception(
    include_top=False, weights="imagenet", input_shape=img_shape, pooling="max"
)
```

```
# for layer in base_model.layers:
#     layer.trainable = False

model = Sequential(
    [
        base_model,
        Flatten(),
        Dropout(rate=0.3),
        Dense(128, activation="relu"),
        Dropout(rate=0.25),
        Dense(4, activation="softmax"),
    ]
)
```



TUMORSENSE

MODEL TRAINING CODE

```
model.compile(
    Adamax(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy", Precision(), Recall()],
)

model.summary()

tf.keras.utils.plot_model(model, show_shapes=True)

"""# 4. Training"""

hist = model.fit(tr_gen, epochs=10, validation_data=valid_gen, shuffle=False)

hist.history.keys()

"""## 4.1 Visualize model performance"""

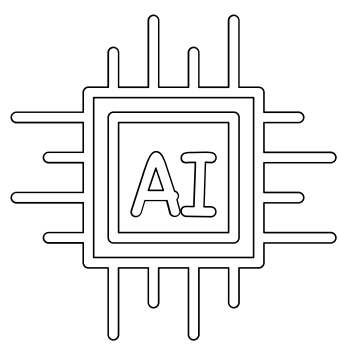
tr_acc = hist.history["accuracy"]
tr_loss = hist.history["loss"]
tr_per = hist.history["precision"]
tr_recall = hist.history["recall"]
val_acc = hist.history["val_accuracy"]
val_loss = hist.history["val_loss"]
val_per = hist.history["val_precision"]
val_recall = hist.history["val_recall"]

index_loss = np.argmin(val_loss)
val_lowest = val_loss[index_loss]
index_acc = np.argmax(val_acc)
acc_highest = val_acc[index_acc]
index_precision = np.argmax(val_per)
per_highest = val_per[index_precision]
index_recall = np.argmax(val_recall)

recall_highest = val_recall[index_recall]

Epochs = [i + 1 for i in range(len(tr_acc))]
loss_label = f"Best epoch = {str(index_loss + 1)}"
acc_label = f"Best epoch = {str(index_acc + 1)}"
per_label = f"Best epoch = {str(index_precision + 1)}"
recall_label = f"Best epoch = {str(index_recall + 1)}"

plt.figure(figsize=(20, 12))
plt.style.use("fivethirtyeight")
```



TUMORSENSE

MODEL TRAINING CODE

```
plt.subplot(2, 2, 1)
plt.plot(Epochs, tr_loss, "r", label="Training loss")
plt.plot(Epochs, val_loss, "g", label="Validation loss")
plt.scatter(index_loss + 1, val_lowest, s=150, c="blue", label=loss_label)
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.subplot(2, 2, 2)
plt.plot(Epochs, tr_acc, "r", label="Training Accuracy")
plt.plot(Epochs, val_acc, "g", label="Validation Accuracy")
plt.scatter(index_acc + 1, acc_highest, s=150, c="blue", label=acc_label)
plt.title("Training and Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

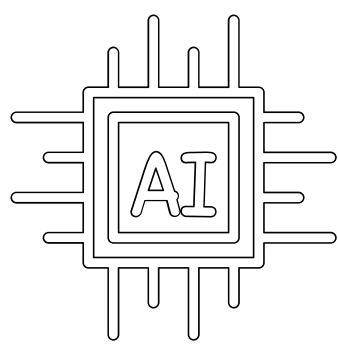
plt.subplot(2, 2, 3)
plt.plot(Epochs, tr_per, "r", label="Precision")
plt.plot(Epochs, val_per, "g", label="Validation Precision")
plt.scatter(index_precision + 1, per_highest, s=150, c="blue", label=per_label)
plt.title("Precision and Validation Precision")
plt.xlabel("Epochs")
plt.ylabel("Precision")
plt.legend()
plt.grid(True)

plt.subplot(2, 2, 4)
plt.plot(Epochs, tr_recall, "r", label="Recall")
plt.plot(Epochs, val_recall, "g", label="Validation Recall")
plt.scatter(index_recall + 1, recall_highest, s=150, c="blue", label=recall_label)

plt.title("Recall and Validation Recall")
plt.xlabel("Epochs")
plt.ylabel("Recall")
plt.legend()
plt.grid(True)

plt.suptitle("Model Training Metrics Over Epochs", fontsize=16)
plt.show()

"""# 5. Testing and Evaluation
```



TUMORSENSE

MODEL TRAINING CODE

```
## 5.1 Evaluate
"""

train_score = model.evaluate(tr_gen, verbose=1)
valid_score = model.evaluate(valid_gen, verbose=1)
test_score = model.evaluate(ts_gen, verbose=1)

print(f"Train Loss: {train_score[0]:.4f}")
print(f"Train Accuracy: {train_score[1]*100:.2f}%")
print("-" * 20)
print(f"Validation Loss: {valid_score[0]:.4f}")
print(f"Validation Accuracy: {valid_score[1]*100:.2f}%")
print("-" * 20)
print(f"Test Loss: {test_score[0]:.4f}")
print(f"Test Accuracy: {test_score[1]*100:.2f}%")

preds = model.predict(ts_gen)
y_pred = np.argmax(preds, axis=1)

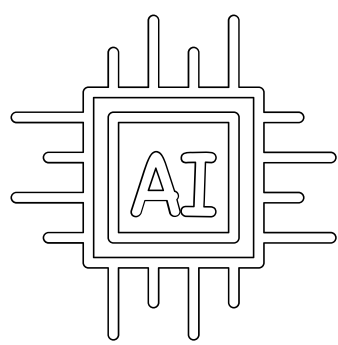
cm = confusion_matrix(ts_gen.classes, y_pred)
labels = list(class_dict.keys())
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels
)
plt.xlabel("Predicted Label")
plt.ylabel("Truth Label")
plt.show()

clr = classification_report(ts_gen.classes, y_pred)
print(clr)

"""## 5.2 Testing"""
```

```
def predict(img_path):
    import numpy as np
    import matplotlib.pyplot as plt
    from PIL import Image

    label = list(class_dict.keys())
    plt.figure(figsize=(12, 12))
    img = Image.open(img_path)
    resized_img = img.resize((299, 299))
    img = np.asarray(resized_img)
    img = np.expand_dims(img, axis=0)
```



TUMORSENSE

MODEL TRAINING CODE

```
img = img / 255
predictions = model.predict(img)
probs = list(predictions[0])
labels = label
plt.subplot(2, 1, 1)
plt.imshow(resized_img)
plt.subplot(2, 1, 2)
bars = plt.barh(labels, probs)
plt.xlabel("Probability", fontsize=15)
ax = plt.gca()
ax.bar_label(bars, fmt="%.2f")
plt.show()
```

```
predict("/kaggle/input/brain-tumor-mri-dataset/Testing/meningioma/Te-meTr_0000.jpg")
```

```
predict("/kaggle/input/brain-tumor-mri-dataset/Testing/glioma/Te-glTr_0007.jpg")
```

```
predict("/kaggle/input/brain-tumor-mri-dataset/Testing/notumor/Te-noTr_0001.jpg")
```

```
predict("/kaggle/input/brain-tumor-mri-dataset/Testing/pituitary/Te-piTr_0001.jpg")
```

```
model.save("AI_MRI_TumorDetect.keras")
```

```
import tensorflow as tf
```

```
# Load your existing Keras model
```

```
model = tf.keras.models.load_model("MRI_BrainTumor_Detection.h5")
```

```
# Create TFLite converter (no quantization)
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

```
# Convert the model
```

```
tflite_model = converter.convert()
```

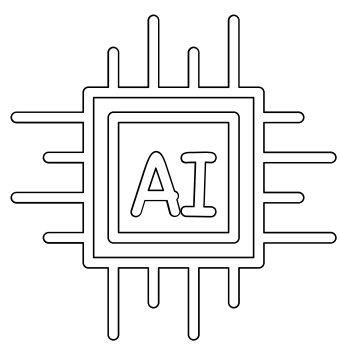
```
# Save the converted TFLite model
```

```
with open("tumor_model.tflite", "wb") as f:
```

```
    f.write(tflite_model)
```

```
print("✅ Model converted and saved as AI_BrainTumorDetector_Final.tflite")
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```



TUMORSENSE

MODEL TRAINING CODE

```
# --- Your Generator Setup ---

batch_size = 32
img_size = (299, 299)

_gen = ImageDataGenerator(rescale=1 / 255, brightness_range=(0.8, 1.2))
ts_gen = ImageDataGenerator(rescale=1 / 255)

# Training generator

tr_gen = _gen.flow_from_dataframe(
    tr_df,
    x_col="Class Path",
    y_col="Class",
    batch_size=batch_size,
    target_size=img_size,
)

# Validation generator

valid_gen = _gen.flow_from_dataframe(
    valid_df,
    x_col="Class Path",
    y_col="Class",
    batch_size=batch_size,
    target_size=img_size,
)

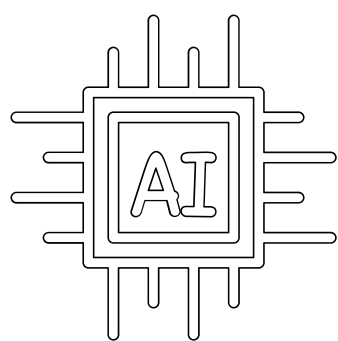
# Test generator

# Test generator

ts_gen = ts_gen.flow_from_dataframe(
    ts_df,
    x_col="Class Path",
    y_col="Class",
    batch_size=16,
    target_size=img_size,
    shuffle=False,
)

# --- ✅ Get classes in correct index order ---

class_dict = tr_gen.class_indices # e.g. {'meningioma': 0, 'glioma': 1, ...}
ordered_classes = [k for k, v in sorted(class_dict.items(), key=lambda item: item[1])]
```

TUMORSENSE

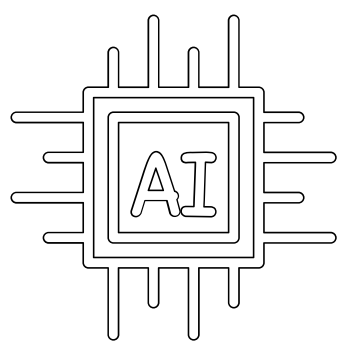
MODEL TRAINING CODE

```
# --- ☒ Save to labels.txt ---  
  
with open("labels.txt", "w") as f:  
    for i, class_name in enumerate(ordered_classes):  
        f.write(f"{i} {class_name}\n")  
print("☒ 'labels.txt' file created successfully with correct class index mapping.")
```

CODE OUTPUT

```
Epoch 1/10  
179/179 ----- 273s 1s/step - accuracy: 0.7094 - loss: 0.7618 - precision: 0.8087 - r  
Epoch 2/10  
179/179 ----- 164s 917ms/step - accuracy: 0.9817 - loss: 0.0632 - precision: 0.9831  
Epoch 3/10  
179/179 ----- 163s 912ms/step - accuracy: 0.9934 - loss: 0.0198 - precision: 0.9937  
Epoch 4/10  
179/179 ----- 164s 913ms/step - accuracy: 0.9952 - loss: 0.0124 - precision: 0.9954  
Epoch 5/10  
179/179 ----- 163s 911ms/step - accuracy: 0.9959 - loss: 0.0137 - precision: 0.9961  
Epoch 6/10  
179/179 ----- 164s 915ms/step - accuracy: 0.9966 - loss: 0.0127 - precision: 0.9967  
Epoch 7/10  
179/179 ----- 168s 941ms/step - accuracy: 0.9952 - loss: 0.0145 - precision: 0.9955  
Epoch 8/10  
179/179 ----- 168s 940ms/step - accuracy: 0.9934 - loss: 0.0178 - precision: 0.9934  
Epoch 9/10  
179/179 ----- 164s 914ms/step - accuracy: 0.9984 - loss: 0.0074 - precision: 0.9984  
Epoch 10/10  
179/179 ----- 203s 918ms/step - accuracy: 0.9990 - loss: 0.0054 - precision: 0.9990
```

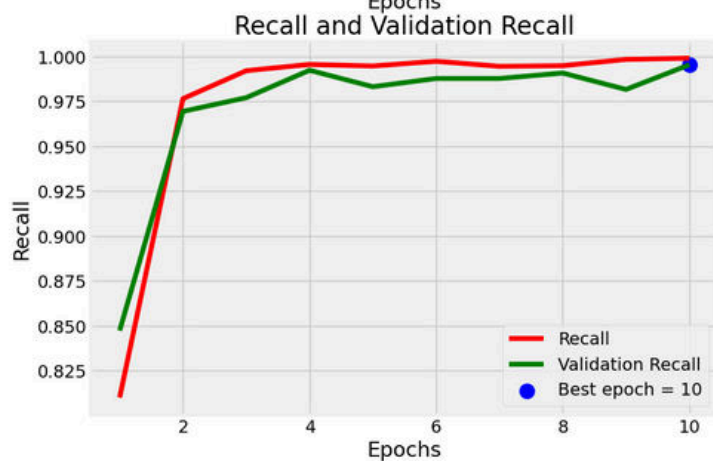
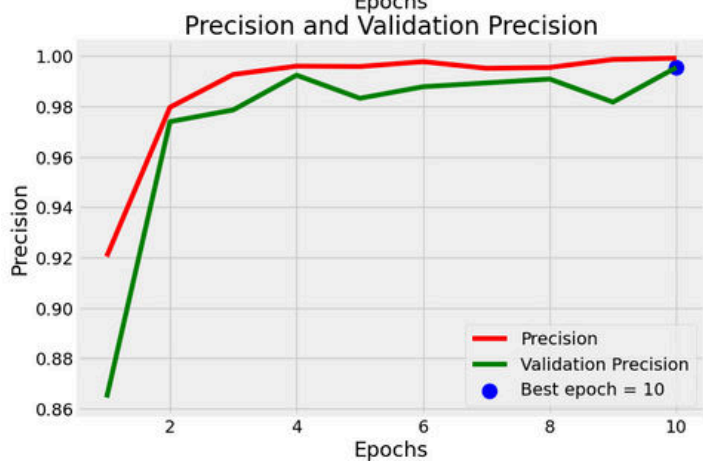
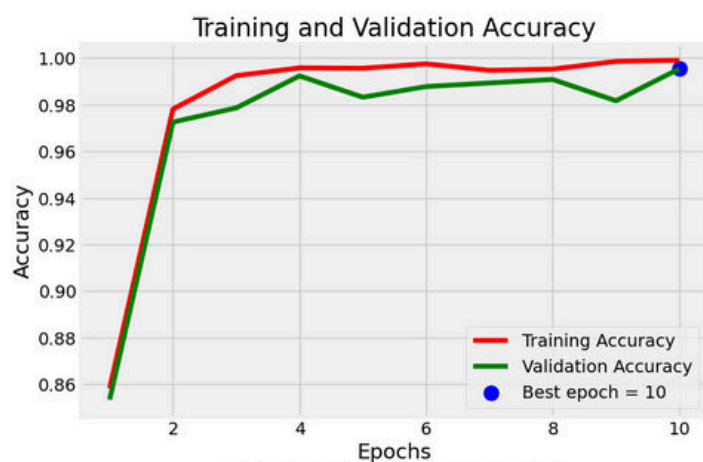
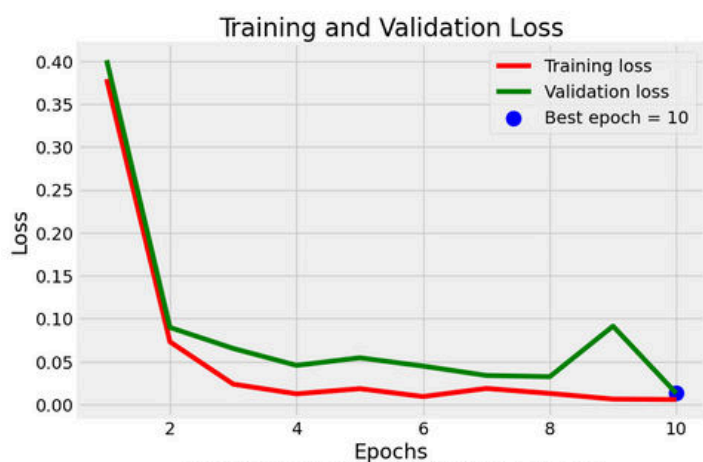
TRAINING INFO PER EPOCH



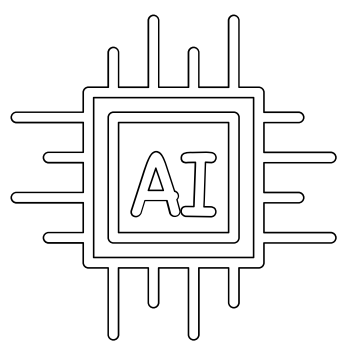
TUMORSENSE

CODE OUTPUT

Model Training Metrics Over Epochs



MODEL TRAINING METRICS



TUMORSENSE

MODEL TRAINING METRICS OVERVIEW

The figure includes four subplots, each comparing training vs. validation performance over 10 epochs. These are critical to understanding model learning behavior and generalization.

TRAINING AND VALIDATION LOSS (TOP-LEFT)

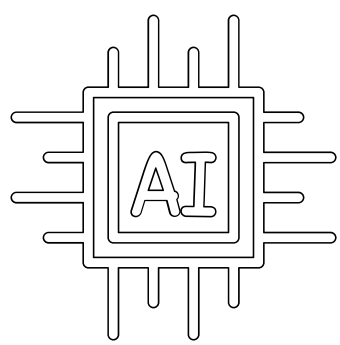
Observation: Both training and validation loss drop steeply in the first few epochs and approach near-zero values by epoch 10.

Interpretation:

The model is **learning well and not overfitting**.

Slight fluctuation in validation loss (especially around epoch 9) is normal.

Best epoch (highlighted as 10) shows lowest overall validation loss.



TUMORSENSE

MODEL TRAINING METRICS OVERVIEW

TRAINING AND VALIDATION ACCURACY (TOP-RIGHT)

Observation: Accuracy increases rapidly and exceeds 98% by epoch 2. Both training and validation accuracies stay high through all epochs.

Interpretation:

The model is highly accurate on both seen (training) and unseen (validation) data.

The consistency between training and validation accuracy indicates good generalization.

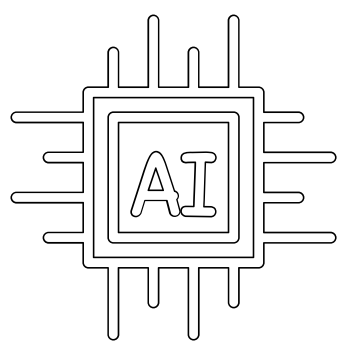
PRECISION AND VALIDATION PRECISION (BOTTOM- LEFT)

Observation: Precision for both training and validation rises steadily and remains high (~98–99%) throughout.

Interpretation:

High precision means few false positives, i.e., the model rarely misclassifies one tumor type as another.

Minor dips are normal and acceptable given the high average performance.



TUMORSENSE

MODEL TRAINING METRICS OVERVIEW

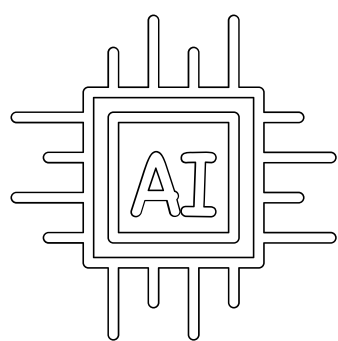
RECALL AND VALIDATION RECALL (BOTTOM-RIGHT)

Observation: Recall increases to **near-perfect (~99%)** by epoch 4 and stabilizes.

Interpretation:

High recall indicates the model is successfully identifying nearly all true tumor cases, minimizing false negatives.

Training and validation recall match closely, indicating low overfitting.

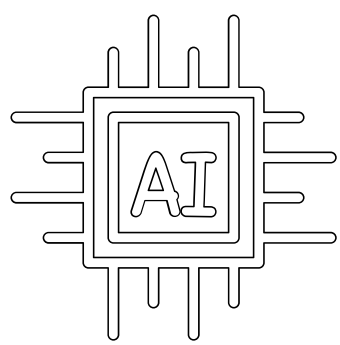


TUMORSENSE

CODE OUTPUT



CONFUSION MATRIX



TUMORSENSE

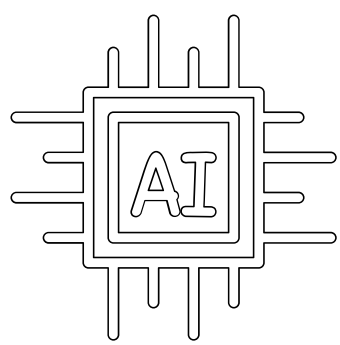
CONFUSION MATRIX ANALYSIS

To evaluate the performance of the brain tumor classification model, the model predictions using a confusion matrix. This matrix gives a clear picture of how well the model is performing across all four tumor classes.

- Rows indicate the actual (true) class labels.
- Columns indicate the predicted class labels by the model.
- Each cell $[i,j]$ shows how many samples from class i were predicted as class j .

INTERPRETATION:

- Glioma: Out of 150 actual glioma cases, the model correctly predicted 148 and misclassified 2 as meningioma.
- Meningioma: Out of 153 cases, 151 were correctly predicted. One was misclassified as glioma and one as pituitary.
- Notumor: The model achieved perfect accuracy (203/203) on this class.
- Pituitary: Out of 150 cases, 149 were correctly predicted and 1 was misclassified as meningioma.



TUMORSENSE

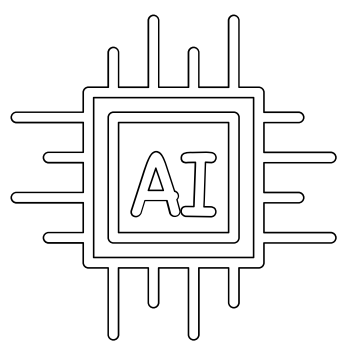
CONFUSION MATRIX ANALYSIS

PERFORMANCE HIGHLIGHTS:

The model performs exceptionally well across all classes.(99% Accuracy)

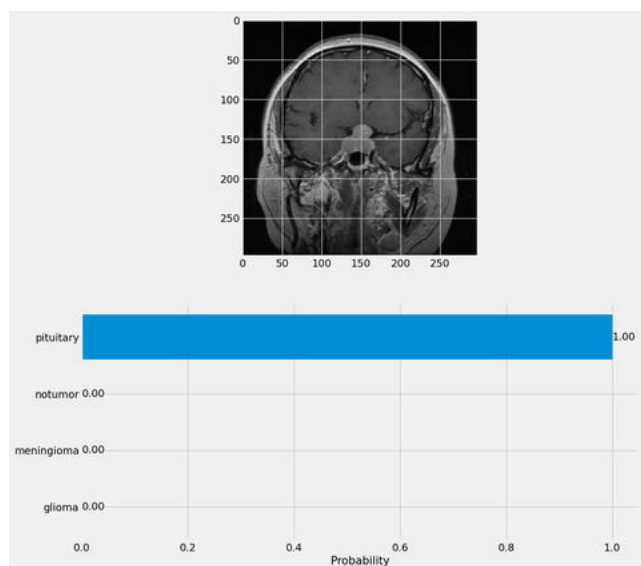
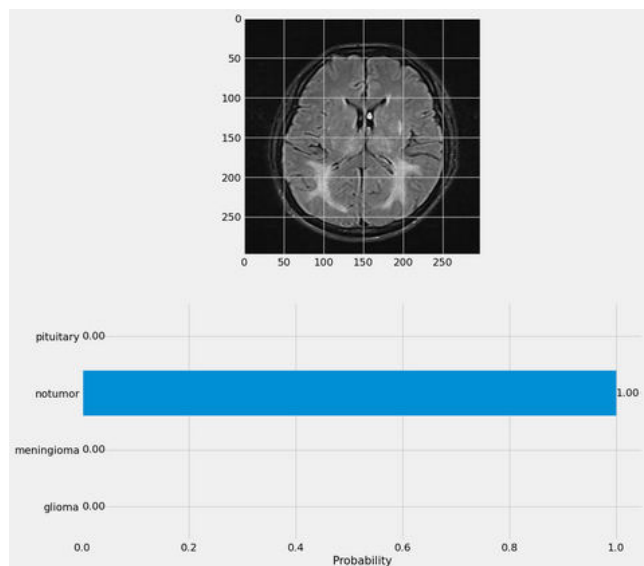
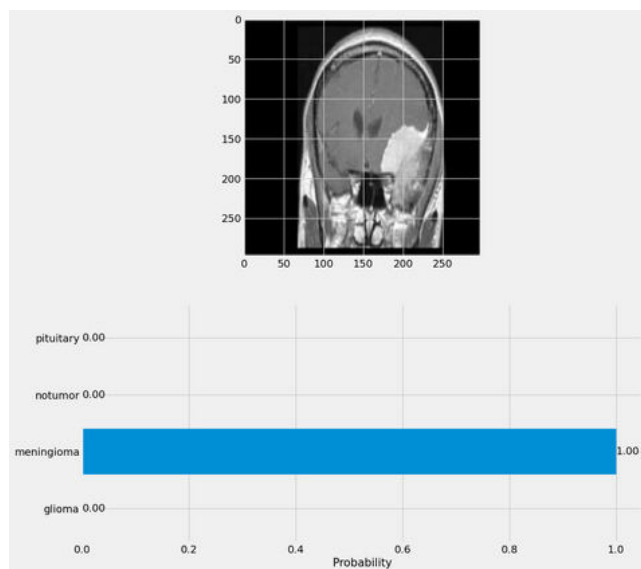
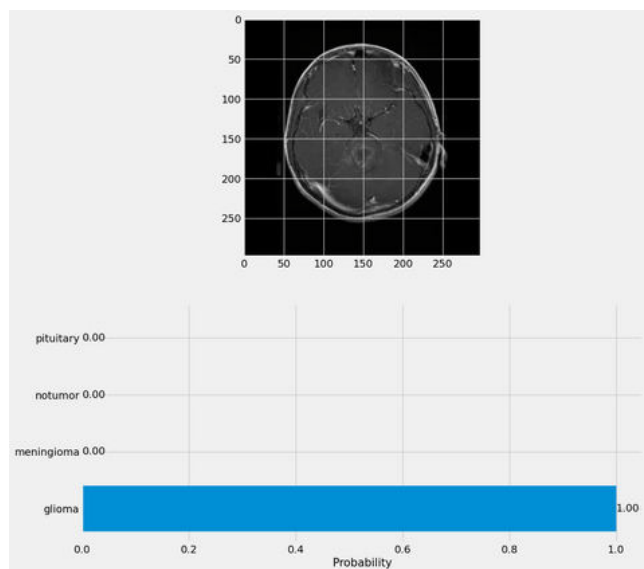
The 'notumor' class achieved 100% accuracy, showing the model's reliability in detecting healthy scans.

Minor misclassifications occurred between glioma, meningioma, and pituitary, which may be due to overlapping features in MRI images.



TUMORSENSE

CODE OUTPUT



TEST PREDICTIONS

100% Accuracy On The Above Tested Images